

Proyecto 2 IPC2

Sistema web de administración de catálogo de biblioteca

Ensayo técnico

Estudiante	José Andrés Díaz Maldonado
Carné	202505653
Curso	Introducción a la Programación y Computación 2
Tecnologías	C#, ASP.NET Core, XML, HTML, CSS, JavaScript y Graphviz

Resumen

El proyecto implementa una aplicación web para gestionar un catálogo jerárquico de categorías y libros. La información se mantiene en memoria mediante listas doblemente enlazadas y nodos desarrollados desde cero. El sistema admite carga incremental desde XML, operaciones manuales desde el navegador y generación de reportes visuales en formato DOT y PNG.

1. Introducción y objetivos

La administración de un catálogo bibliográfico exige representar relaciones jerárquicas entre categorías y mantener libros ordenados para facilitar búsquedas y consultas. El proyecto resuelve este problema con una aplicación ASP.NET Core y una interfaz web completamente en español.

El requisito central consiste en evitar las colecciones genéricas de .NET para el almacenamiento del catálogo. Por ello, las relaciones dinámicas se construyen con referencias entre nodos propios. Las bibliotecas nativas se utilizan solamente para funciones externas como la lectura sintáctica de XML, la atención de solicitudes web y la ejecución del generador de imágenes.

Objetivo general

Desarrollar un sistema web en C# que permita administrar categorías jerárquicas y libros mediante TDA propios, cargar información desde XML y generar reportes gráficos verificables.

Objetivos específicos

- Implementar nodos y listas doblemente enlazadas sin `List<T>`, diccionarios ni LINQ.
- Registrar categorías principales y subcategorías con nombres únicos.
- Insertar libros ordenados ascendentemente por ISBN y permitir búsqueda, eliminación y consulta de vecinos.
- Procesar múltiples archivos XML sin eliminar los datos existentes, salvo que el usuario solicite reiniciar el catálogo.
- Exportar la jerarquía y los libros de una categoría como código DOT e imágenes PNG.
- Proporcionar una interfaz web para ejecutar todas las operaciones del catálogo.

Alcance

El catálogo permanece en memoria mientras el servidor está en ejecución. El XML funciona como medio de entrada, no como base de datos permanente. Al cerrar la aplicación, la información cargada se pierde y debe volver a ingresarse o importarse.

2. Arquitectura y estructuras propias

La aplicación separa modelos, estructuras, servicios e interfaz web. **Inicio** configura ASP.NET Core y publica las rutas de la API. **GestorCatalogo** concentra las reglas de negocio. **LectorXml** procesa archivos de entrada y **GeneradorGraficos** crea los reportes.

Modelos reales

- **Libro:** almacena ISBN, título, autor y nombre de categoría.
- **Categoría:** almacena nombre, referencia al padre, lista de hijos y lista de libros.
- **Resultado, ResultadoCarga y ResultadoGrafico:** comunican el estado y los mensajes de cada operación.

TDA implementados

ListaCategorias enlaza objetos **NodoCategoria** mediante referencias *Siguiente* y *Anterior*. Conserva referencias a *Primero*, *Ultimo* y la cantidad de elementos. Cada categoría contiene otra lista del mismo tipo para representar sus hijos, por lo que la jerarquía puede crecer dinámicamente.

ListaLibros utiliza objetos **NodoLibro** y mantiene los libros ordenados por ISBN desde el momento de inserción. La eliminación ajusta las referencias vecinas y actualiza los extremos de la lista cuando corresponde.

Reglas de negocio

- Los nombres de las categorías son únicos sin distinguir mayúsculas y minúsculas.
- Una subcategoría solo puede registrarse cuando existe su categoría padre.
- El ISBN debe contener únicamente números y no puede repetirse.
- Cada libro debe pertenecer a una categoría existente.
- La búsqueda y eliminación recorren manualmente las listas de todas las categorías.
- Reiniciar elimina la referencia inicial del catálogo y permite que el recolector de memoria libere los nodos inaccesibles.

Complejidad principal

Operación	Complejidad
Agregar categoría al final	$O(1)$
Buscar categoría	$O(c)$
Insertar libro ordenado	$O(l)$
Buscar o eliminar libro	$O(c + l)$
Consultar ISBN menor y mayor	$O(n)$
Vaciar catálogo	$O(1)$

En la tabla, *c* representa las categorías visitadas, *l* los libros de una lista y *n* todos los libros del catálogo.

3. Carga incremental de XML

El servicio **LectorXml** utiliza `XmlReader` y `XmlDocument` únicamente para interpretar el documento. Después de la lectura, todos los datos se transfieren a los TDA propios.

Formato admitido

```
<config>
<listaCategorias>
<categoria>Ciencia</categoria>
<categoria padre="Ciencia">Física</categoria>
</listaCategorias>
<listaLibros>
<libro>
<ISBN>978001</ISBN>
<titulo>Fundamentos de física</titulo>
<autor>Ana López</autor>
<categoria>Física</categoria>
</libro>
</listaLibros>
</config>
```

Proceso de carga

- Se valida que el elemento raíz sea `config`.
- Las secciones `listaCategorias` y `listaLibros` son opcionales.
- Las categorías se procesan en pasadas sucesivas; esto permite que una hija aparezca antes que su padre dentro del archivo.
- Las categorías e ISBN existentes se contabilizan y no se duplican.
- Los libros nuevos se validan y se insertan en la lista de la categoría correspondiente.
- Una relación circular o un padre inexistente detiene la carga con un mensaje claro.
- La opción “Vaciar el catálogo antes de cargar” permite reemplazar toda la información.

Seguridad de lectura

El lector prohíbe DTD, deshabilita resoluciones externas y limita el documento a diez millones de caracteres. Estas medidas reducen riesgos al procesar archivos suministrados desde la interfaz.

4. Aplicación web e integración

La interfaz se sirve desde `wwwroot` y se comunica con rutas HTTP definidas en **Inicio.cs**. El navegador envía formularios y presenta las respuestas sin recargar la página completa.

Operaciones disponibles

Ruta	Función
GET /api/catalogo	Mostrar categorías y libros
POST /api/categorias	Registrar categoría
POST /api/libros	Registrar libro
GET /api/libros/{isbn}	Buscar libro
GET /api/libros/{isbn}/vecinos	Consultar ISBN menor y mayor
DELETE /api/libros/{isbn}	Eliminar libro
POST /api/cargar	Procesar archivo XML
POST /api/reiniciar	Vaciar el catálogo
GET /api/reportes/{tipo}	Generar reporte solicitado

Reportes gráficos

GeneradorGraficos recorre directamente *NodoCategoria* y *NodoLibro* para construir la sintaxis DOT. El sistema ejecuta el comando `dot`, guarda el código fuente y produce una imagen PNG dentro de `wwwroot/reportes`.

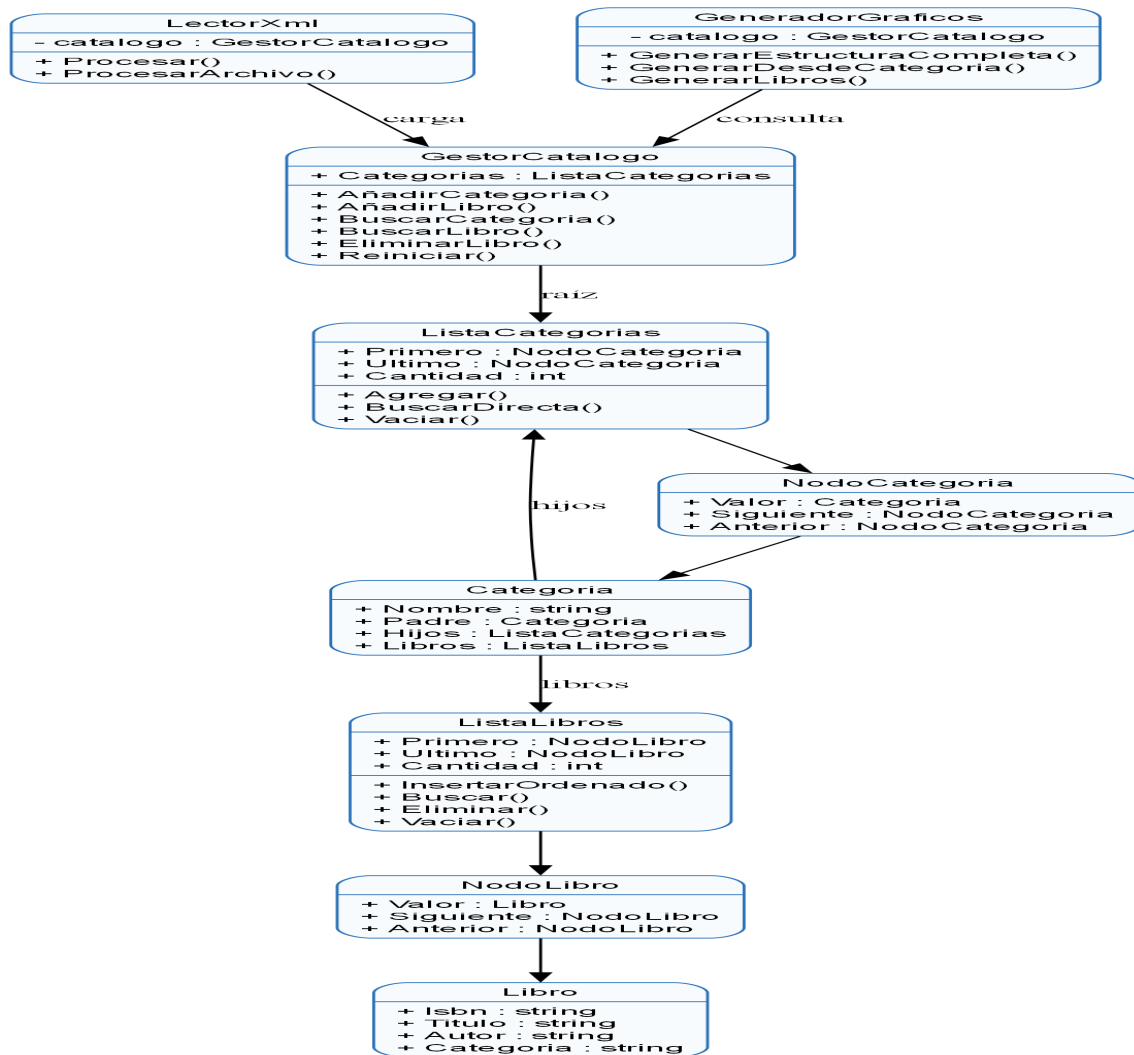
- **Estructura completa:** muestra todas las categorías y relaciones padre-hijo.
- **Vista desde una categoría:** representa la categoría seleccionada y todos sus descendientes.
- **Libros de una categoría:** muestra los libros en el orden ascendente que conserva `ListaLibros`.

Los nombres de archivo incorporan fecha y hora para evitar colisiones. La interfaz permite abrir el código DOT y descargar la imagen generada.

Validación integrada

La versión final se verificó mediante compilación sin advertencias, carga del XML de ejemplo, consulta de libros y vecinos, registro y eliminación, generación de los tres reportes y acceso HTTP tanto a los archivos DOT como a las imágenes PNG.

5. Diagrama de clases actualizado



El diagrama representa las clases y asociaciones presentes en el código entregado. GestorCatalogo posee la lista raíz; cada Categoria contiene sus hijos y libros; las listas se forman con nodos doblemente enlazados.

6. Conclusiones

- 1 Los TDA propios satisfacen el requisito de administrar memoria dinámica mediante referencias entre nodos sin recurrir a colecciones genéricas.
- 2 La jerarquía recursiva permite agregar niveles de subcategorías sin establecer una profundidad fija.
- 3 La carga incremental facilita distribuir categorías y libros entre varios archivos XML y conserva la información previa.
- 4 La interfaz web reúne todas las operaciones del catálogo y expone mensajes claros ante datos inválidos.
- 5 Los reportes DOT y PNG permiten comprobar visualmente las relaciones del catálogo y el orden de los libros.

Referencias

Microsoft. Documentación de ASP.NET Core y System.Xml. <https://learn.microsoft.com/dotnet/>
Graphviz. Documentación del lenguaje DOT. <https://graphviz.org/documentation/>
Repositorio del proyecto.
https://github.com/joseandresdiazmaldonado4-art/IPC2_Proj02_202602_202505653